Lab 3

Encoding Data as Visual Channels · CSC316, Fall 2026

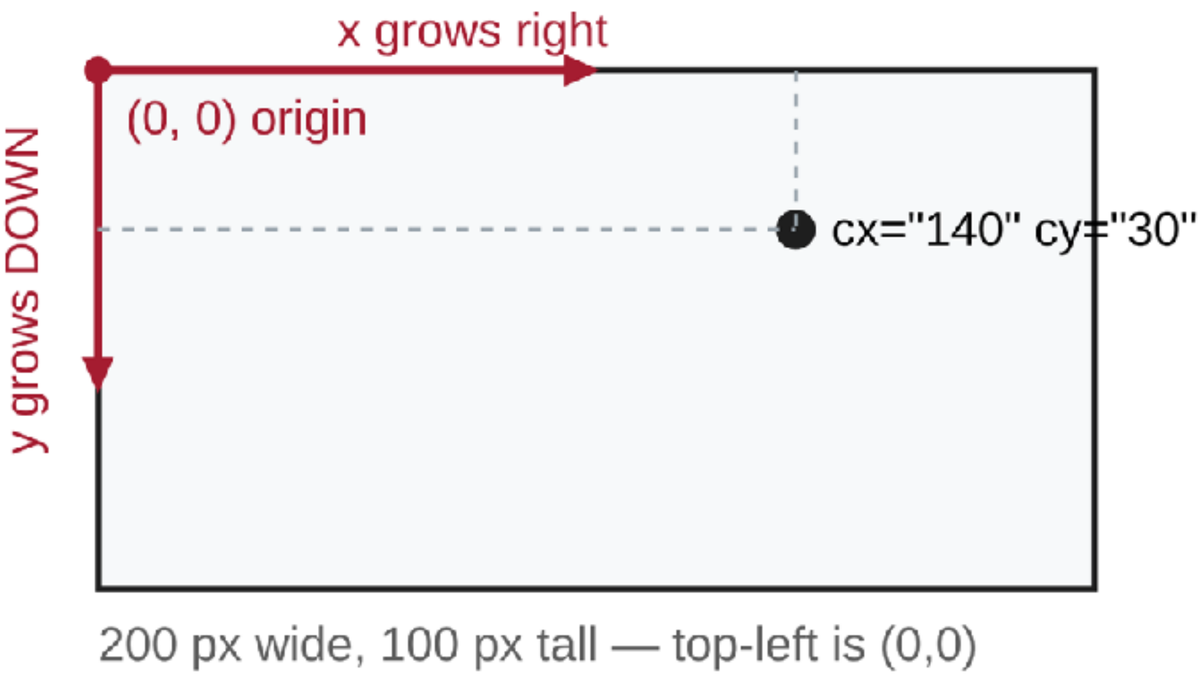
PART ONE

Drawing with SVG



Think of the **SVG** as the drawing area, SVG elements (“marks") are named shapes you can add to your drawing.

The coordinate system



200 px wide, 100 px tall — top-left is (0,0)

width and height set the box, in pixels:

```
<svg width="200" height="100">
  <circle cx="140" cy="30"
          r="6" />
</svg>
```

The component takes the data

`Scatter.svelte` — the data arrives as a prop:

```
<script>
  let { data } = $props();
</script>
<svg width="430" height="150">
  {#each data as d}
    <circle cx={d.x} cy={d.y} r="7" />
  {/each}
</svg>
```

The component takes the data

`Scatter.svelte` — the data arrives as a prop:

```
<script>
  let { data } = $props();
</script>
<svg width="430" height="150">
  {#each data as d}
    <circle cx={d.x} cy={d.y} r="7" />
  {/each}
</svg>
```

data = { {x: 1, y: 1},
{x: 3, y: 2},
{x: 6, y: 2.5} }



<Scatter data={[3 rows]} /> → 3 circles

The component takes the data

`Scatter.svelte` — the data arrives as a prop:

```
<script>
  let { data } = $props();
</script>
<svg width="430" height="150">
  {#each data as d}
    <circle cx={d.x} cy={d.y} r="7" />
  {/each}
</svg>
```

data = { {x: 1, y: 1},
{x: 2, y: 2},
{x: 3, y: 1.5},
{x: 4, y: 2.5},
{x: 5, y: 1.8},
{x: 6, y: 2.8} }



<Scatter data={[6 rows]} /> → 6 circles, new coordinates

Data values are not pixels

Three rows of data:

```
{ x: 1,   y: 2   }
{ x: 1.8, y: 0.6 }
{ x: 3,   y: 2.4 }
```

The drawing area:

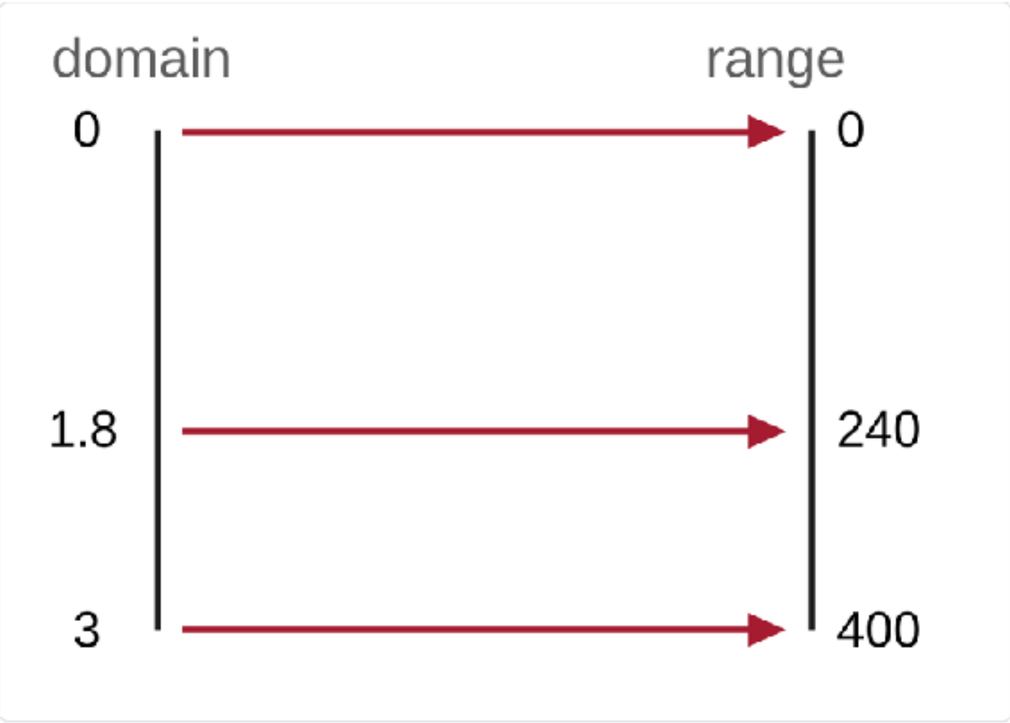
```
400 px wide, 300 px tall
```

A scale does the conversion

A scale is a function: data in, pixels out.

```
const xScale = d3.scaleLinear()
  .domain([0, 3])     // data units
  .range([0, 400]);   // pixels

xScale(0)    //   0
xScale(1.8)  // 240
xScale(3)    // 400
```



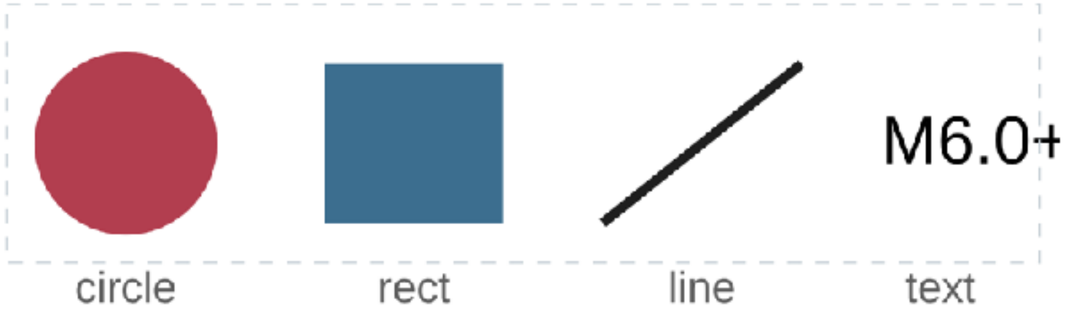
Equal steps in the data become equal steps on screen. For y, flip it: `.range([300, 0])`.

Five elements

Five elements draw almost everything:

```
<circle cx cy r />
<rect   x y width height />
<line   x1 y1 x2 y2 />
<text   x y>M6.0+</text>
<g transform="translate(40,20)">
  …a group, moved together
</g>
```

What they look like:



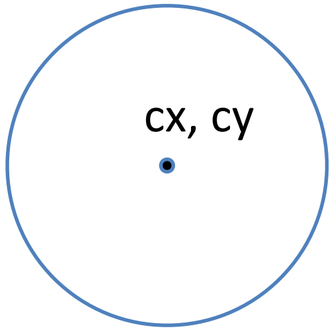
the dashed box: one <g> holding all four

Every channel is an attribute

CHANNEL	SVG ELEMENT AND ATTRIBUTE	GOOD FOR
Position x, y	`<circle cx cy>`, `<rect x y>`	quantitative; the most precise comparisons
Length	`<rect width>` or `height`	quantitative, from a zero baseline
Size / area	`<circle r>`	magnitude, roughly compared
Color	`fill`	categories (hue) or a sequence (light to dark)
Shape	`<circle>` vs `<rect>` vs `<path>`	a few categories

One column, one channel, one attribute. A scale computes the attribute's value.

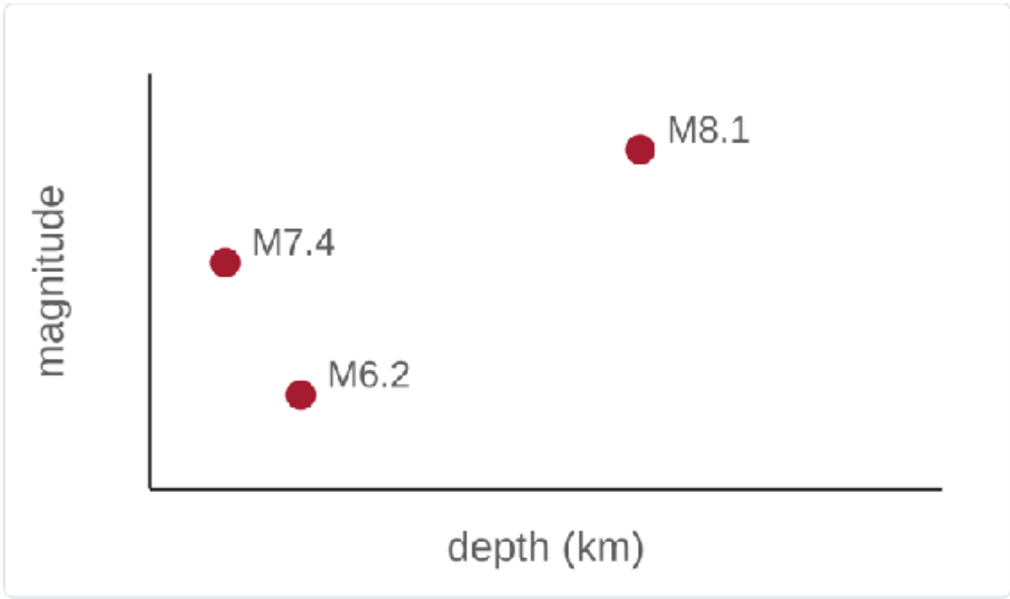
Example: position for <circle>



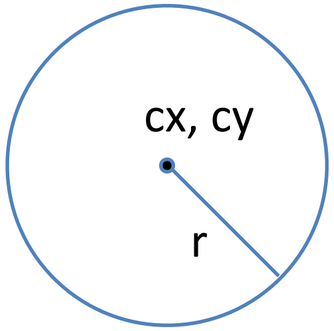
One row becomes one mark:

```
{#each quakes as d}
  <circle cx={xScale(d.depth)}
          cy={yScale(d.mag)}
          r="6" />
{/each}
```

`depth` → x position, `mag` → y position.



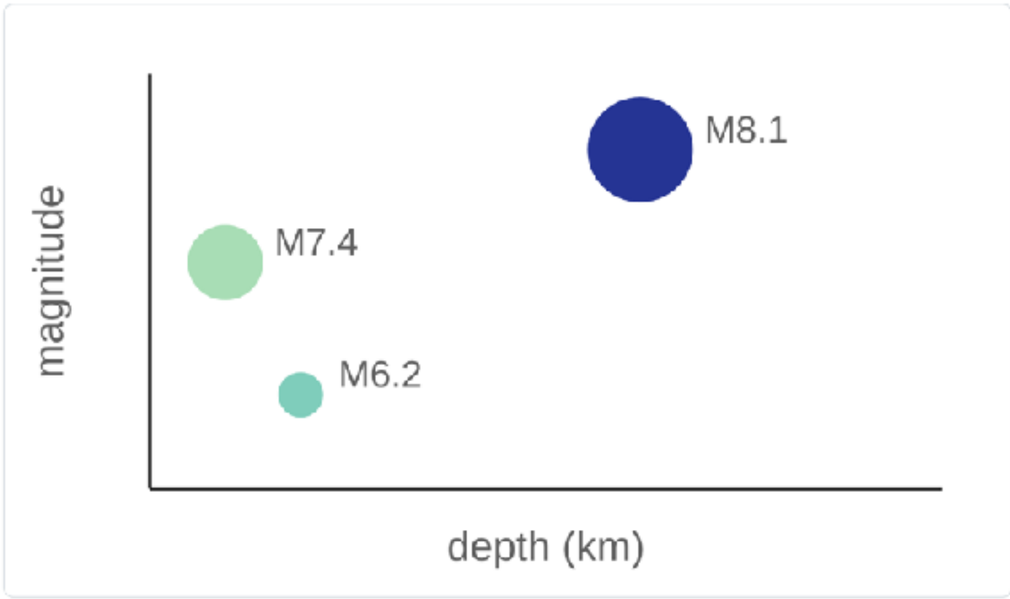
Example: size and color for <circle>



Two more channels, two more attributes:

```
<circle cx={xScale(d.depth)}
        cy={yScale(d.mag)}
        r={rScale(d.mag)}
        fill={cScale(d.depth)} />
```

`mag` → size, `depth` → color. The marks did not change; two attributes did.



PART TWO

Today's lab



The data

- 155 quakes, M6.0 and larger, one year, from the USGS.
- longitude, latitude, depth, mag, magType.
- Also place and time, for tooltips and stretch goals.

A new project this week

- npx sv create quake-encoding, then npm install d3.
- npm run dev, and leave it running.
- git init, git add -A, git commit.

Turn in

- Deliverables by 11:59 PM, October 7
- Take screenshots of your running SvelteKit project (terminal + localhost in the url)

Stretch goals

- Encode magType as shape. Compare and reflect.
- Add a tooltip with place, mag and depth.
- Color or animate by time: do quakes come in bursts?
- Plot a magnitude histogram at the M6.0 cutoff.